AI-Driven Quality Assistance Application for STLC Automation

A PROJECT REPORT
SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENTS FOR THE DEGREE OF

Master of Technology (Online)
IN
Data Sciences and Business Analytics

BY

Monika Tyagi



Faculty of Engineering
Indian Institute of Science
Bangalore – 560 012 (INDIA)

November, 2026

Declaration of Originality

I, **Monika Tyagi**, with SR No. **13-19-02-19-52-24-1-24515** hereby declare that the material presented in the project report titled

AI-Driven Quality Assistance Application for STLC Automation

represents original work carried out by me at **Alight Services India Pvt Ltd** as part of the project credit requirements of the **Master of Technology (Online)** degree in **Data Science and Business Analytics** at the Indian Institute of Science, between **August, 2024** to **November, 2026**.
With my signature, I certify that:

- I have not manipulated any of the data or results.
- I have not committed any plagiarism of intellectual property. I have clearly indicated and referenced the contributions of others.
- I have explicitly acknowledged all collaborative research and discussions.
- I have understood that any false claim will result in severe disciplinary action.
- I have understood that the work may be screened for any form of academic misconduct.

Date: Student Signature

In our capacities as internal project guide and faculty mentor of the above-mentioned work, we certify that the above statements are true to the best of our knowledge, and we have carried out due diligence to ensure the originality of the report.

Internal Guide Name: *Kapil Madaan* Internal Guide Signature
Organisation: *Alight Services India Pvt Ltd*

Faculty Mentor Name: *Punit Rathore* Faculty Mentor Signature
Department: *Cyber Physical Systems*

© Monika Tyagi

November, 2026

All rights reserved

Acknowledgements

It is my pleasure to thank Prof. Punit Rathore, who served as the faculty mentor of the project at IISc, for his continuous support and guidance throughout this project. His insights helped me understand the way to look at the problem and take decisions at various stages of the project. Whenever I encountered any difficulty during implementation, he would take out time to guide me and suggest ways out of those situations. His inputs were very important in determining the overall design and course of this project.

I am also grateful to Prof. Deepak Subramani for guiding me through the entire process of completion of the project and preparation of the report. His inputs helped me in organizing the project, focusing on certain aspects of evaluation and presentation of the result. His guidance made the process of completion of the project and documentation easier.

I would also like to thank Mr. Kapil Madaan, who served as the company mentor for this project, for his help and guidance during the design and development process of the system. His practical suggestions helped me in taking decisions at various technical levels. His guidance was of immense help whenever I encountered any difficulty during implementation.

I would also like to thank my family and friends for their support during this journey of the M.Tech. programme.

Abstract

The Software Testing Life Cycle (STLC) comprises many activities, including requirements understanding, test planning, test design, test execution, defect tracking, and release decisions. Most teams use a distinct tool for each of them, which makes manual transfer of information between the activities necessary. This project looks into the use of AI to unite these activities within one orchestrator that uses a collection of specialised agents. The approach should help teams turn requirements, user stories, code changes, and test results into quality insights, without automating the review of results or the approval of releases.

The prototype is a web application built with React and TypeScript, FastAPI, and PostgreSQL, with a separate agent service implemented using Google's Agent Development Kit (ADK) for all AI-related tasks. At the core is a root agent that determines the type of each request and passes it to one of nine specialised agents covering test-case generation, test-data generation, test automation assistance, test execution analysis, test self-healing, change impact analysis, defect reporting, test summary creation, and release readiness checking. The application can import Jira Cloud stories and obtain code changes from GitHub, and it supports optional Microsoft Entra single sign-on and different model providers.

This report covers the motivation, related research, architecture, implementation, security, and evaluation. It distinguishes the functionality in the prototype from what is proposed as future work: the test execution analysis agent works with the test evidence available but does not run external test suites, and self-healing recommendations are suggestions only. No final benchmark or performance gain is reported. Whether the system is efficient, reliable, and suitable for practical use has to be shown by future testing and evaluation.

Contents

Acknowledgements i

Abstract ii

Contents iii

List of Figures vi

List of Tables vii

List of Algorithms viii

Acronyms, Abbreviations and Symbols x

1 Introduction 1

1.1 Background 1

1.2 Problem Statement and Motivation 2

1.3 Research Questions 3

1.4 Objectives 3

1.5 Scope and Contributions 4

1.6 Report Organization 5

2 Literature Review 6

2.1 Software Testing Life Cycle 6

2.2 Existing Practices and Tools 7

CONTENTS

2.3 LLMs in Software Quality Engineering . . . 8
2.4 Related Work on AI-Assisted Software Testing . . . 8
2.5 Agent-Orchestrated Workflows . . . 9
2.6 Grounding, Traceability, and Human Review . . . 9
2.7 Evaluation of AI-Assisted Testing . . . 10
2.8 Research Gaps . . . 10

3 Methodology and Implementation **12**
3.1 Development Method . . . 12
3.2 Requirements and Actors . . . 13
3.3 Data and Inputs . . . 13
3.4 Technology Stack . . . 15
3.5 System Architecture . . . 16
3.5.1 Frontend . . . 16
3.5.2 Backend and Persistence . . . 17
3.5.3 Agent Orchestration . . . 17
3.5.4 Assistance Request Processing . . . 20
3.5.5 Human-in-the-Loop Refinement . . . 21
3.5.6 External Integrations and Model Providers . . . 21
3.6 Security and Data Handling . . . 21
3.7 Deployment and Operations . . . 23
3.8 Quality Attributes . . . 23

4 Results and Discussions **24**
4.1 Prototype Implementation Status . . . 24
4.2 Existing Automated Test Assets . . . 24
4.3 Demonstration Scenarios . . . 24
4.3.1 Interface Walkthrough and Illustrative Outputs . . . 26
4.4 Evaluation Plan . . . 30
4.4.1 Dataset and Protocol . . . 31

CONTENTS

4.4.2 Measures . . . 31
4.4.3 End-to-End Time: One Week to One Hour . . . 32
4.4.4 Metrics for Each Specialist . . . 33
4.4.5 Orchestration and Workflow Metrics . . . 35
4.4.6 Human Evaluation . . . 35
4.4.7 Benchmark Dataset . . . 35
4.4.8 Analysis and Validity . . . 36
4.5 Limitations . . . 37

5 Conclusions **38**
5.1 Future Work . . . 39

References **40**

List of Figures

3.1 High-level design of the Quality Assistance application. 16

3.2 Illustrative agentic workflow showing orchestration among the root agent and specialist agents. 18

3.3 Sample sequence of requests via the application services. 20

4.1 Dashboard workspace where a quality assistance request is initiated. 26

4.2 Settings page where Jira, GitHub, and sign-in are configured. Availability is contingent upon the configuration made by the administrator and valid credentials. 27

4.3 Example request input containing quality context and a user story before agent processing. 27

4.4 Example of the response generated along with test case recommendations. . . . 28

4.5 Example of the test data suggested as part of the response generated. 28

4.6 Example of the automation notes and test script recommendations. 29

4.7 Example of release risk and readiness analysis based on the submitted context. . 30

List of Tables

2.1 Examples of existing tools by STLC activity. 7

3.1 Selected functional requirements and corresponding implemented behavior. . . . 13
3.2 Inputs to an assistance request. The combined text is limited to 70,000 characters. 14
3.3 Technology stack used in the prototype. 15
3.4 Registered specialist agents and the scope described by their instructions. 19

4.1 Implementation status as of November 2026, distinguishing current behavior from planned work. 25
4.2 Proposed evaluation measures; no values are reported because a controlled evaluation has not yet been completed. 31
4.3 Matrix for the one-week to one-hour claim. Manual and assisted entries are estimates or targets; the measured column is to be filled in after the evaluation runs. 33
4.4 Proposed main metric for each specialist. No values are reported because the evaluation has not been carried out. 34
4.5 Proposed orchestration and workflow metrics. 35
4.6 Proposed pilot benchmark. The sizes are planning figures. 36

List of Algorithms

No explicit algorithms beyond standard library implementations are used in this project. The core inference logic is delegated entirely to the Google ADK agents and the configured LLM.

Acronyms, Abbreviations and Symbols

These short forms are used in more than one chapter.

Acronyms

ADK Agent Development Kit

API Application Programming Interface

CI/CD Continuous Integration and Continuous Delivery

LLM Large Language Model

M.Tech. Master of Technology

MCP Model Context Protocol

QA Quality Assurance

RQ Research Question

STLC Software Testing Life Cycle

UI User Interface

Abbreviations

E2E End-to-end

JWT JSON Web Token

ACRONYMS, ABBREVIATIONS AND SYMBOLS

RAG Retrieval-Augmented Generation

REST Representational State Transfer

Chapter 1

Introduction

1.1 Background

Running tests is just one of the aspects of software quality. Within the Software Testing Life Cycle (STLC), the team reads the requirement, determines the scope, designs test cases and test data, runs tests, registers defects, creates a summary, and, finally, often in a hurry, approves or rejects the release. According to ISO/IEC/IEEE 29119 and the ISTQB Foundation syllabus, this process has to be planned and traceable to risk [1, 2].

The step which used to cause delays, in my experience, was not any particular step. It was the dispersion among tools. There is a story reported as a ticket. There is a change made as a code difference. There is a run which produced a defect logged in the log. One has to take the ticket and convert it to test cases, define what to regress against the code difference, and produce some information about the run that is understandable to the release manager. It is an ordinary step for one ticket. It is the repetition of the same interpretation process across a sprint, and the wording varies from ticket to ticket.

Large language models (LLMs) handle the linguistic aspect of the process quite well. They can summarise the ticket, suggest a few boundary cases, and even take the messy log and fill the defect template. There is already a sizeable body of work on using them for testing [3]. They have problems when there is no one to look at them. I have seen models skipping a rule in the second paragraph, adding a field which never existed, and still being confident. For the

quality team, this is a problem. An assistant is good only when it uses the context provided by the user, concentrates on its task, shows the draft, and waits for confirmation or rejection by the user.

1.2 Problem Statement and Motivation

The major rationale for undertaking this project is the need for an intelligent and connected approach to handle various tasks during the Software Testing Life Cycle (STLC). Most of the time, the quality-related activities are done independently, and the information often has to be looked at and interpreted once again. While there has been a lot of improvement in automation and simplifying of individual testing tasks, the processes of requirement analysis, defect identification, and release readiness are still quite manual. Such fragmentation may cause certain scenarios to be overlooked, efforts to be duplicated, and possible risks to be identified much later in the development cycle.

To resolve such issues, this project proposes an AI-assisted approach that brings the STLC activities together in one place using a centralised orchestrator. Users submit the requirements and user stories, and either the source code modifications or the test execution results, depending on the activity. The root agent evaluates the request and distributes the task among the specialist agents accordingly. The results obtained by these agents are consolidated into one response for the quality team to make a decision.

The system is developed to help engineers and not to make decisions for them. It assists in the preparation and organisation of testing-related data and gives suggestions which can be taken after their analysis. The validation of test cases, execution of tests in the proper environment, validation of defects, and release decisions remain with the concerned engineers themselves.

1.3 Research Questions

The project is guided by the following research questions:

1. **RQ1:** How can a multi-agent AI system coordinate several STLC activities through one quality-assistance interface?
2. **RQ2:** What validation, provenance, and human-review controls are needed before AI-generated testing artifacts can be trusted enough to use?
3. **RQ3:** How should the system be evaluated for its effect on requirement-to-test traceability, quality-engineering effort, coverage, and defect analysis?

1.4 Objectives

The main objectives of this project are to:

- design and develop a full-stack application where users can submit software-testing information and review the results generated with AI assistance;
- build a central agent that can understand user requests and delegate tasks to specialised agents covering different activities of the STLC;
- provide secure, authenticated access to the application and maintain user-specific records of submitted requests and their results;
- integrate with engineering tools such as Jira for accessing user stories and GitHub for retrieving source-code changes;
- allow users to configure different AI model providers and application deployment settings;
- support human-in-the-loop refinement, so that a user can review a generated result, add further instructions, and rerun it until it is acceptable; and
- establish a practical and reproducible evaluation approach in which the generated results are reviewed by humans, without making claims about improvements that have not yet been measured.

1.5 Scope and Contributions

This report deals with the code of the project repository [4] as of November 2026. It should be noted that what is submitted here is a prototype. In no way am I proposing a new theory of agents. The browser interface, the API and the agent service work together. Users are able to copy and paste requirements and stories, attach git code differences or the link to the GitHub pull request with the changes, add the test log, continue the conversation in a later turn and export the response. If Jira Cloud and GitHub are set up, issue text and pull-request or branch code differences can be fetched instead of typed. The root agent then works with nine STLC specialists.

Compared with the existing practices in Chapter 2, the contributions of this work are:

- one orchestrated workflow, in which a root agent routes a request to nine STLC specialists and combines what they return into a single response;
- shared context across activities, since requirements, stories, code differences, and test results are handled together and a session identifier lets later turns build on earlier answers;
- connections to the engineers' own tools, with Jira Cloud for user stories and GitHub for code differences;
- human-in-the-loop operation, where the user can refine a result with further guidance or send a request to one specialist only, while every decision stays with the engineer;
- a configurable design with several model providers, authentication, and records kept per user; and
- an evaluation protocol with effort-based measures for the assisted workflow.

These are engineering contributions. The report does not claim a new algorithm or a measured improvement.

There are, of course, limitations that I do not wish to omit. The prototype will not execute the tests of another software suite. It will not transform scripts in a git repository. It cannot

confirm that the tests produced are exhaustive, nor can it sign off on the release. Execution, self-healing, and readiness responses are only suggestions based on the text received. I also did not incorporate retrieval from a test repository, CI/CD execution, output schema enforcement, or a comparative study; these remain future work.

1.6 Report Organization

Chapter 2 is the reading I used: STLC, LLMs in testing, agent setups, and the validation issues. Chapter 3 is how I built the prototype, including architecture and security. Chapter 4 is what I can actually show, plus the evaluation I still need for the research questions. Chapter 5 is the wrap-up and the list of next steps.

Chapter 2

Literature Review

2.1 Software Testing Life Cycle

Software testing is an important part of software development. It involves a series of planned activities to check whether a software product works as expected and to identify areas where improvements may be needed. The ISO/IEC/IEEE 29119 series provides guidelines for software testing, including its processes, concepts, and documentation. Similarly, the ISTQB Foundation syllabus explains the key activities involved in testing, such as planning, analysing requirements, designing test cases, preparing tests, executing them, and completing the testing process [1, 2]. Both approaches emphasise the importance of maintaining traceability throughout testing. This means that test cases and their results should be linked to the original requirements and identified risks, so that quality-related decisions can be supported by actual evidence.

In practice, the Software Testing Life Cycle (STLC) does not always follow a fixed sequence of steps. The outcome of one activity can influence what happens next. For example, testing may reveal gaps or unclear points in the requirements, while identifying a defect may require additional regression test cases. Similarly, changes in the source code can affect which parts of the application need to be tested again. Because of these dependencies, an effective system should be able to retain information from earlier stages and use it when supporting later activities. Rather than generating separate outputs for each testing task, an AI-assisted system should help maintain the connection between different stages and make it easier for engineers

to understand how one activity relates to another.

2.2 Existing Practices and Tools

In most teams each STLC activity has its own tool, and people carry the information from one tool to the next. Table 2.1 lists some examples. It is only a sample of what is out there, not a feature-by-feature comparison, and it does not say that any tool lacks a capability.

STLC activity	Example tools	What the tool offers
Requirements, stories, defects	Jira, Azure DevOps	Tracking of requirements, user stories, and defects.
Test design	Katalon, testRigor, Diffblue	Katalon drafts test cases from a requirement, a Jira ticket, or a prompt and leaves them for the tester to review [5]; testRigor lets teams write tests in plain English [6]; Diffblue creates unit tests automatically [7].
Test data	Tonic.ai	Synthetic and de-identified data for testing [8].
Automation and execution	Selenium, Playwright, Cypress, Appium, mabl	Scripted browser and mobile automation; mabl uses agents to create, run, analyse, and maintain tests [9].
Test maintenance	Katalon, Applitools	Self-healing tests and visual checks [5, 10].
Test management and reporting	TestRail, Zephyr, Xray, Katalon	Test-case management; Katalon keeps requirements, test cases, evidence, defects, and releases in one record [5].

Table 2.1: Examples of existing tools by STLC activity.

Many of these products now have AI features, connect to trackers such as Jira, and keep a person in the review step. They are still mostly organised around one activity or one product, and the public descriptions do not show how context travels from requirements, through code changes and test results, to a release decision. Teams also use general-purpose AI assistants, usually in an ad hoc way, for example to draft a test case or to explain a failure.

In my organisation also, test design, automation, and release reporting are handled through a combination of Jira for tracking requirements and defects, and automation scripts for test automation. The process involves drafting test cases from requirements, executing them, and

reviewing the results before making release decisions. This workflow is largely manual, with team members responsible for ensuring that tests align with business requirements and that any issues are properly documented and addressed.

2.3 LLMs in Software Quality Engineering

LLMs can read a story and write scenarios. They can suggest data, classify a failure, or summarise a run. That is why people reach for them: the jobs are repetitive and mostly text. Reliability is a different matter. A model may ignore a constraint, invent a step, or write as if it is sure when the input is thin.

Amershi et al. make a similar point for machine learning in software teams: the workflow has to stay with people, ownership has to be clear, data quality has to be watched, and the system has to be monitored after it ships [11]. I treated that as something I had to design for. In this assistant the model output is a draft. A tester should be able to check the assumptions, edit the cases, and drop a recommendation that is not supported, before it goes into a plan or a release discussion.

2.4 Related Work on AI-Assisted Software Testing

A few recent surveys give a good picture of where AI in testing stands. Wang et al. went through 102 studies on LLMs in testing and found that preparing test cases and repairing programs are the tasks people tackle most. They also look at which models and prompting styles are used [3]. Faraji and Pombo take a tool-oriented view of AI-driven test automation, covering techniques, tools, and open challenges [12]. Further back, Fontes and Gay mapped 124 publications on machine learning for test generation. The problems they list, such as training data, scalability, evaluation, and repeatability, are worth keeping in mind for LLM-based work too [13].

For test design from requirements, Hasan et al. built a dataset that pairs use cases with high-level test cases and tried GPT-4o, Gemini, LLaMA 3.1 8B, and Mistral 7B. Fine-tuning helped the two smaller models, and the people who rated the generated cases were positive about them. In their survey of practitioners, the hard part was rarely writing scripts. It was keeping the testing in line with the business requirements [14]. That is why the test-case

specialist in this work starts from requirements and user stories.

On the architecture side, Lahami and Krichen review how GenAI-based testing systems are put together [15], which is the closest work to the multi-agent design used here. Most of the studies above look at one task at a time, test generation above all. What this report adds is not a better generator. It is a way to connect several STLC activities in one application, with a person reviewing the output.

2.5 Agent-Orchestrated Workflows

The usual agent pattern is a set of specialists and a coordinator that decides who to call. ReAct is one way of mixing reasoning with actions. Broader surveys talk about planning, tools, memory, and how agents are coordinated [16, 17]. Splitting the work made prompts easier for me to maintain. It did not make the answers correct. More calls also mean more latency and more cost, and the coordinator can pick the wrong specialist or call too many of them.

My prototype has a root agent and a registry of STLC specialists. Routing depends on the request and the files or text attached to it. When I read the code for this report, that routing was still mostly the instructions plus whatever the model generated. The split helped me implement the thing. It is not a workflow engine, and nobody has verified it formally. The next step I identified is schemas between stages, some validation, and a place where a person has to approve.

2.6 Grounding, Traceability, and Human Review

Retrieval-Augmented Generation (RAG) is the usual answer when a model needs facts it should not invent. Lewis et al. retrieve documents and then generate from them [18]. In testing that would mean using approved requirements, policies, API specs, or old defects, instead of only the pasted prompt. Right now the prototype uses what the user attached, and whatever is still sitting in the in-memory session.

Even with retrieval, an evaluation has to tell two things apart: whether a case can be traced to a requirement, and whether the case is actually right. People still have to catch gaps and unsupported claims. If the output could affect a release, the evidence should stay on screen,

uncertainty should be visible, and someone should have to accept it.

2.7 Evaluation of AI-Assisted Testing

When I designed the evaluation, I wanted a baseline, inputs that look like real tickets, a frozen model and prompt, and scores that can be repeated. The measures I chose are requirement-to-test links, scenario coverage, human-rated accuracy, time on the same tasks, defect classification where labels exist, and end-to-end latency. They are not interchangeable. A longer list of cases is not better coverage. A faster first draft is not less work if review and correction take the time back.

NIST's AI Risk Management Framework organises this around governance, mapping, measurement, and management [19]. For me that meant: record the model and config versions, write down how a reviewer should score an artifact, keep suggestions separate from verified results, and treat privacy and operational risk as part of the evaluation, not something to add later.

2.8 Research Gaps

Looking at the practices and tools above, I found several possible gaps. They are questions to investigate, not proven absences.

- **Fragmented lifecycle.** Tools tend to specialise in one activity, so the same requirement, change, or failure can end up being re-entered and re-interpreted at each stage.
- **Context across phases.** The public tool descriptions do not make clear how context is shared between requirements, test design, code changes, execution, and release assessment.
- **Change-aware testing.** A code change can mean that existing tests need another look, and this is not always tied to test design and release decisions.
- **Joined-up reporting.** Information sits in separate outputs, so a single summary and release view still takes manual effort.

- **Human oversight.** AI-generated artefacts need review, and the review loop should be part of the workflow and not an afterthought.
- **Thin evidence.** There is little measured evidence on effort and quality when AI help is applied across the whole STLC with human review. The surveys also point to open problems in evaluation and benchmarks [13].

This work responds to the first five with one orchestrated application, specialised agents, and a refine-and-rerun step for the engineer. For the sixth it sets out an evaluation protocol. It does not claim that no other platform offers a similar workflow, and it does not claim better results than existing tools, because that comparison has not been done yet (Chapter 4).

Chapter 3

Methodology and Implementation

3.1 Development Method

The project was carried out in an iterative manner, starting with identifying the challenges faced by quality teams, followed by designing and developing a working prototype. The work involved planning the overall architecture, building the frontend and backend services, integrating the AI agents, and identifying how the completed system could be evaluated. This approach was suitable for the project, as the main outcome was a working application that brings together different activities of the Software Testing Life Cycle.

The initial proposal outlined a phased approach, beginning with public and open-source projects to explore the system's capabilities, with the possibility of using anonymised organisational data in a controlled setting at a later stage.

To understand the actual state of the system, the implementation was reviewed across its main components, including the frontend, backend, agent service, test cases, and deployment documentation. The description of implemented features in this report is based on the functionality available in the code and the workflows that can currently be accessed through the application.

3.2 Requirements and Actors

The person I had in mind is a quality engineer, a developer, a business analyst, or someone on a release call who wants structured testing help, not another open-ended chat. After login they provide requirements or stories, maybe code differences or a test-run log, and then they read the reply. A follow-up can reuse a session id. An administrator picks the model and turns Jira or GitHub on through environment settings.

Requirement	**Prototype behavior**
Context submission	Accepts requirements, user stories, code differences, and optional session context through the authenticated assistance API.
Quality assistance	Returns a generated response that can include prioritized cases, test data, automation notes, and release risks.
Specialized analysis	Routes suitable requests to registered STLC agents for test design, execution-log analysis, change impact, defects, summaries, and readiness.
User separation	Associates assistance requests with the authenticated user and persists request inputs and responses in PostgreSQL.
Engineering context	Offers Jira Cloud story retrieval and GitHub pull-request or branch code-difference retrieval when configured.
Operational configuration	Supports provider/model selection, containerized local startup, health checks, and production configuration checks.
Human-in-the-loop refinement	Once a result is generated, the user can add instructions, optionally focus the rerun on a single specialist, and rerun in the same session, repeating this until the result is acceptable.

Table 3.1: Selected functional requirements and corresponding implemented behavior.

3.3 Data and Inputs

The application does not train or fine-tune any model, so there is no training dataset to describe. The models are used as the provider supplies them, and every answer is produced from the context that comes with the request. Table 3.2 lists what a request can carry.

Preprocessing is kept light on purpose. The backend trims the text, drops empty optional fields, rejects control characters, applies the size limits, and runs a keyword check that turns away submissions with no software or testing context. It then builds one message with sections for the requirements, user stories, code differences, and any review guidance. That is followed

Input	Where it comes from	Size limit
Requirements (required)	Typed or pasted by the user. Test-run logs and notes are also pasted here.	20,000 characters
User stories (optional)	Typed, or imported from Jira Cloud issues assigned to the user.	20,000 characters
Code differences (optional)	A pasted git difference, or fetched from a GitHub pull request or branch comparison.	50,000 characters
Review guidance (optional)	Instructions the user types to refine an earlier result.	5,000 characters
Target specialist (optional)	One of the nine specialists, chosen by the user.	not applicable
Session identifier	Returned by the agent service and reused for follow-up turns.	not applicable

Table 3.2: Inputs to an assistance request. The combined text is limited to 70,000 characters.

either by the specialist the user asked for or by a request for prioritised test cases, test data, automation notes, and release risks. There is no chunking, embedding, or retrieval step.

3.4 Technology Stack

Table 3.3 lists the technologies used in the prototype and the role each one plays.

Layer	Technology	Role
Frontend	React 19, TypeScript, Vite	User interface, component rendering, and build tooling.
Routing	React Router	Client-side navigation between login, workspace, settings, and guide pages.
Markdown rendering	react-markdown, remark-gfm	Renders the formatted agent response, including tables, in the browser.
HTTP client	Browser Fetch API	Calls from the browser to the backend REST API.
Backend API	FastAPI (Python), Uvicorn	REST endpoints, request validation, and routing.
Agent orchestration	Google Agent Development Kit (ADK)	Root agent, nine specialised agents, tool use, and session handling.
LLM access	Gemini, LiteLLM, Amazon Bedrock, Ollama	Configurable model providers used by the agents for text generation.
Database	PostgreSQL	Users, assistance requests, and Jira and GitHub connections.
ORM and migrations	SQLAlchemy (async), Alembic	Data access and database schema migrations.
Authentication	JWT (python-jose), bcrypt	Access tokens and password hashing; Microsoft Entra single sign-on is optional.
Encryption	Cryptography (Fernet)	Encrypts stored Jira and GitHub credentials.
External APIs	HTTPX	Asynchronous calls from the backend to the agent service, Jira Cloud, and GitHub.
Deployment	Docker, Docker Compose	Runs the frontend, backend, agent service, and database together.
Configuration	Environment variables (`.env`)	Provider selection, credentials, and production settings.

Table 3.3: Technology stack used in the prototype.

3.5 System Architecture

There are four pieces: the browser UI, the API, the database, and the agent process. The React and TypeScript frontend calls the FastAPI backend. After the user is authenticated, the backend stores the request, builds the agent message, and later stores the reply. A separate service runs the root agent and the specialists through Google ADK. PostgreSQL holds users, requests, and the integration records.

Figure 3.1 describes the high-level design of the web application on AWS.

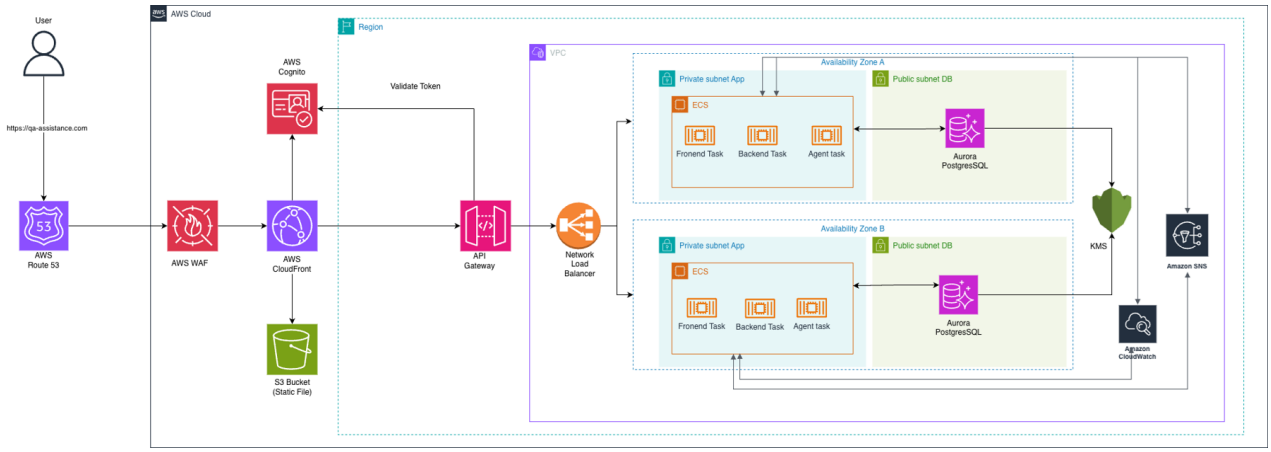


Figure 3.1: High-level design of the Quality Assistance application.

3.5.1 Frontend

The UI has login, a dashboard for requests, a settings page, and a short user guide. On the dashboard the user can type requirements and stories, paste a change, or start from the samples I added: smoke planning, an end-to-end example, change impact, execution and reporting, automation, and a follow-up. If Jira is connected, assigned issues can fill the story field. If GitHub is connected, a pull-request or compare code difference can be fetched. The reply is rendered in the browser. It can also be downloaded as Markdown or HTML. Under each result there is a refine-and-rerun panel, where the user can type further instructions, pick one specialist to focus on, and run the request again in the same session.

3.5.2 Backend and Persistence

The backend is FastAPI, with asynchronous SQLAlchemy on PostgreSQL [20]. `/api/assist` needs a logged-in user. The request is stored with owner, source text, session id, status, timestamps, and later the response. Then the backend builds a structured message and calls the agent service. Both success and failure are written back. Errors go to the frontend on the normal error path.

Users, assistance requests, Jira connections, and GitHub repository connections are separate tables. That is enough to open an old submission. It is not enough if you expect the agent conversation to survive a restart. The ADK runner I used keeps session state in memory.

3.5.3 Agent Orchestration

The root agent is `quality_assistance_agent`. I built it with Google ADK, the configured model, logging callbacks, and the tools available at start-up [21]. On the supported cloud providers the root agent can pass work to the registered sub-agents. The backend prompt asks for prioritized cases, suggested data, automation notes, and release risks, using the requirements, stories, and code differences that came in.

3. METHODOLOGY AND IMPLEMENTATION

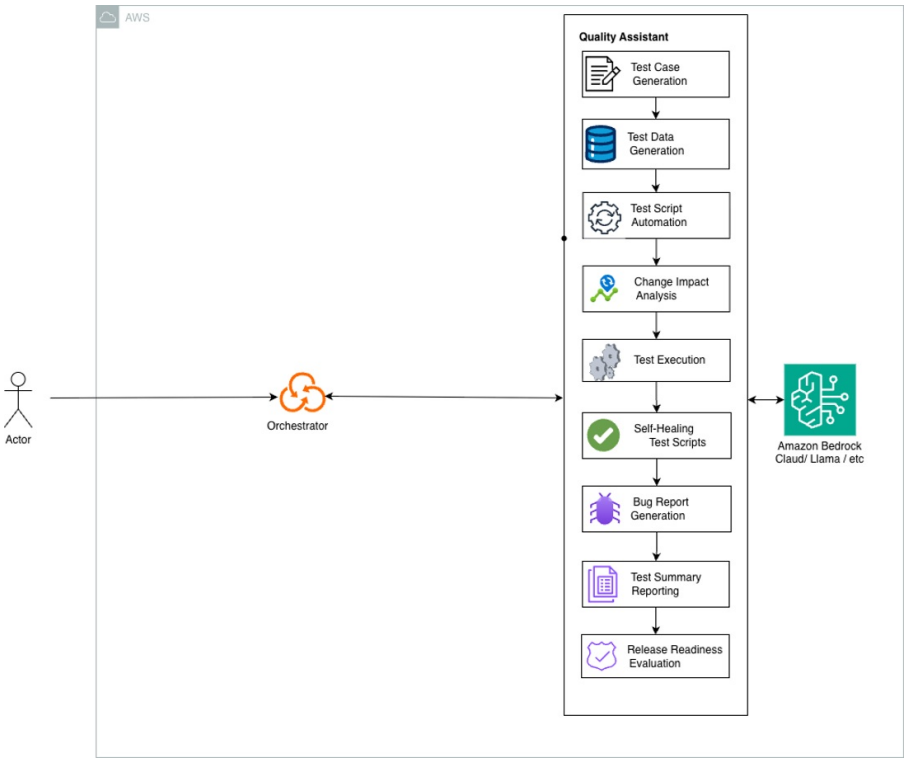


Figure 3.2: Illustrative agentic workflow showing orchestration among the root agent and specialist agents.

Requirement and story text go to the test-case and test-data agents. Code differences go to change impact analysis. Execution logs go to execution, defect, and summary. Release questions go to readiness. The root agent also has a small requirement-summarization tool. All of this is prompt routing. There is no second engine underneath it that would force the same path every time.

ID	Specialist	Implemented responsibility
S01	Test case generation	Drafts structured, prioritized cases from requirements and stories, including positive, negative, and boundary scenarios.
S02	Test data generation	Suggests representative and edge-case test data.
S03	Test script automation	Suggests automation approaches and repeatable script outlines.
S04	Test execution	Interprets supplied or simulated test-run results and highlights failures; it does not itself launch the user's test suite.
S05	Self-healing test scripts	Recommends maintenance strategies when selectors or interfaces change; it does not directly edit repository files.
S06	Change impact analysis	Relates supplied code differences to potentially affected areas and regression scope.
S07	Bug report generation	Structures failure information into an actionable defect report.
S08	Test summary reporting	Summarizes test outcomes for stakeholders.
S09	Release readiness evaluation	Weighs supplied test and risk signals and provides a reasoned recommendation.

Table 3.4: Registered specialist agents and the scope described by their instructions.

3.5.4 Assistance Request Processing

The steps for processing an assistance request are:

1. Validate the request and associate it with the authenticated user.
2. Persist the request inputs (requirements, optional stories, and code differences) with status `processing`.
3. Construct a message from the requirements, stories, and code differences, adding any review guidance or the specialist the user chose.
4. Resolve or create an agent session for the user.
5. Invoke the root agent with the message and session.
6. Route to one or more specialists according to the request context.
7. Collect the generated response from the agent events.
8. If the agent call succeeds, persist the session identifier, response, and status `completed`; otherwise persist the failure status and return an explicit service error.
9. Return the request identifier, session, response, and status.

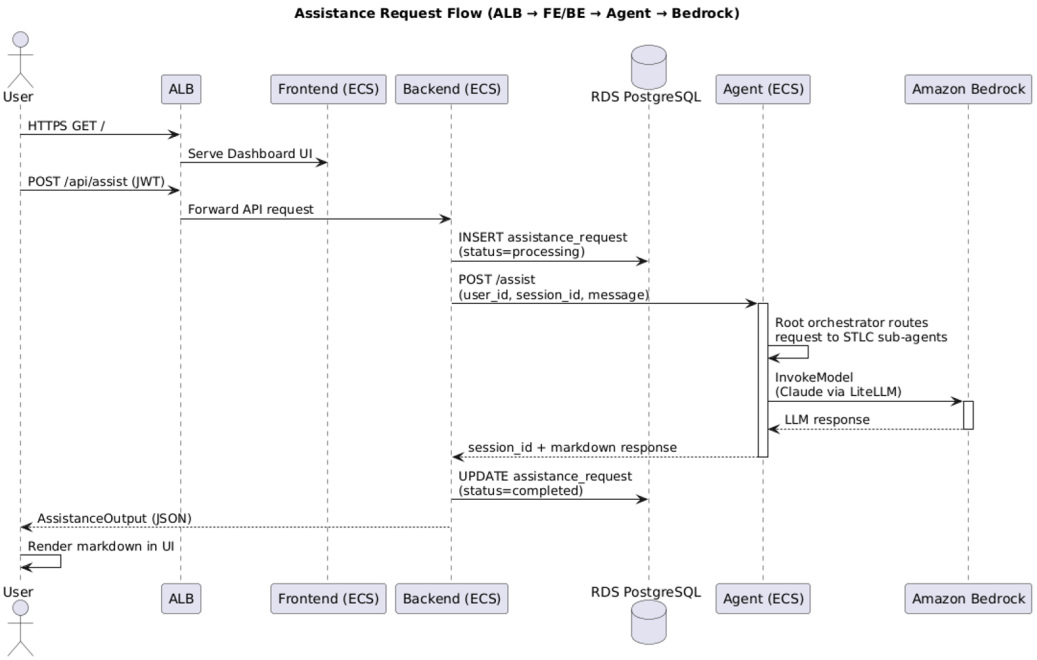


Figure 3.3: Sample sequence of requests via the application services.

3.5.5 Human-in-the-Loop Refinement

I designed the application to treat every response as a draft. After a result comes back, the engineer reads it and can add more input or instructions, for example to add negative cases, shorten a verdict, or change how a table is laid out. The backend appends those instructions to the original context as a review section. If the user picks a specialist, the request is limited to that one agent. The session identifier is reused, so the agent still has the earlier turns in front of it. This cycle of generate, review, instruct, and rerun can be repeated as often as needed, and every decision to accept a result stays with the engineer.

3.5.6 External Integrations and Model Providers

Jira Cloud is OAuth 2.0, a short-lived state record, and Jira APIs to list assigned issues and fetch detail. GitHub settings store a repository and an encrypted personal access token. From there the backend can pull pull-request code differences or compare two references. Microsoft Entra SSO is optional. The agent service can also act as a Model Context Protocol (MCP) client and attach optional Jira and GitHub tools from an MCP server, when one is configured [22]. The import flow described above does not use MCP. The agent side can use Gemini, LiteLLM, Amazon Bedrock, or Ollama if the credentials and models are there. I put those options in so the same code can run in more than one place. I did not benchmark every provider on the same tasks.

3.6 Security and Data Handling

Assistance records are scoped to the signed-in user. JWT tokens are issued after register or login. Passwords go through the project's password manager. Jira and GitHub credentials are stored encrypted. Jira OAuth state is a digest and it expires. In production the settings check expects real secrets, an encryption key, explicit HTTPS origins, database TLS, and non-default database credentials. The ECS note describes private app and database services, IAM roles, and managed secrets.

Requirements, stories, code differences, and test results are often confidential. The application stores them with the request and sends them to the configured model provider. Anyone

who deploys this has to pick a provider and a retention policy the organisation will accept, limit who can see stored data, rotate credentials, and keep secrets out of the prompt. The checks I added reduce accidental leaks. They are not a privacy assessment, a threat model, or an internal security sign-off.

3.7 Deployment and Operations

For local use there is a Docker Compose file with PostgreSQL, frontend, backend, and agent. There are also scripts if you want the UI and Python services on the host, with the database local or in a container. The AWS ECS Fargate note is a pilot layout: separate containers, RDS PostgreSQL, an Application Load Balancer, and a chosen model. It is a checklist. It is not Terraform or CloudFormation, and I am not claiming that an AWS production stack has been brought up.

3.8 Quality Attributes

- **Modularity:** frontend, API, database, and agent run as separate processes. Specialists are registered on their own.
- **Configurability:** model, endpoints, integrations, and production flags come from the environment.
- **Traceability:** each user's inputs and replies are stored. I still do not have a formal requirement-to-case trail, and agent memory is not durable.
- **Availability:** health and readiness endpoints are there, and the container notes help. How well a real install holds up depends on the topology someone actually chooses.
- **Reliability:** the backend has error paths and settings checks. Generated test artifacts still need a person to read them, and they still need a proper study.

Chapter 4

Results and Discussions

4.1 Prototype Implementation Status

What I have at this point is a running full-stack prototype. It is more than a slide architecture. A user can register, sign in, send context through the authenticated API, and read a reply on the dashboard. Requests are stored. Nine specialists are registered, and the root agent picks among them from its routing instructions. GitHub code-difference retrieval, Jira Cloud OAuth and issue fetch, and optional Microsoft Entra login are in the same GitHub repository [4].

4.2 Existing Automated Test Assets

I wrote focused tests for Bedrock credentials, backend production settings, Jira Cloud behaviour, and Microsoft Entra SSO. Those files show that a few paths mattered enough to get checks. They do not tell you the suite is green today, and they do not cover the UI, the model, the database, and the agents end to end. If a later version of this report is going to claim test evidence, it should include the command, the environment, the commit, the test count, and the actual result of a run.

4.3 Demonstration Scenarios

The dashboard includes samples I used while building, so I could hit several agent routes without inventing a ticket every time:

Project area	**Status in inspected repository**	**Evidence or qualification**
Problem analysis and architecture	Implemented	Research framing is documented in the proposal and presentation; current services are separated into frontend, backend, and agent.
Core quality-assistance flow	Implemented	Authenticated request persistence, prompt construction, agent call, and response persistence are present.
Specialist STLC capabilities	Implemented	Nine task-focused agents are registered; orchestration is controlled by ADK and agent instructions.
Jira and GitHub context	Implemented, configuration-dependent	OAuth and issue retrieval are available for Jira; GitHub repository settings support code-difference retrieval.
Authentication and deployment controls	Implemented in part	Password/JWT authentication, optional Entra SSO, encrypted integration credentials, and production settings validation are present.
RAG over organizational test assets	Planned	No persistent requirements/test/defect retrieval index is evidenced in the inspected implementation.
Automated CI test execution and script mutation	Planned / out of scope of current behavior	Execution and healing agents analyze or recommend; they do not execute external suites or commit changes.
Comparative empirical evaluation	Pending	No measured baseline, benchmark results, or controlled organizational pilot results were supplied.

Table 4.1: Implementation status as of November 2026, distinguishing current behavior from planned work.

1. a password-reset requirement, for planning and summarising;
2. an e-commerce checkout with requirements, stories, and code differences;
3. an authentication change with large code differences, for regression impact;
4. a test-run log with pass, fail, and skipped cases, for execution analysis and reporting;
5. a Playwright selector change, for automation and self-healing notes; and
6. a follow-up that reuses the previous session id.

4.3.1 Interface Walkthrough and Illustrative Outputs

The screenshots are one ordinary path: open the workspace, check settings, type a requirement and a story, then read the reply. They show the screens and a sample output.

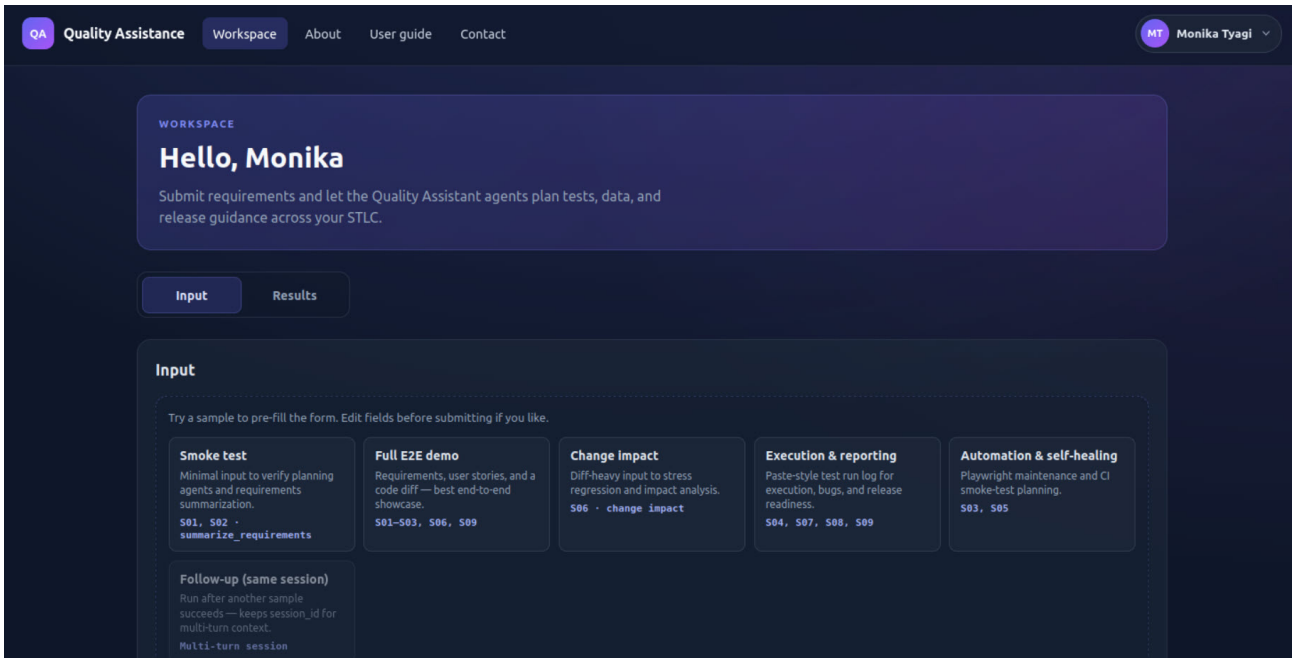


Figure 4.1: Dashboard workspace where a quality assistance request is initiated.

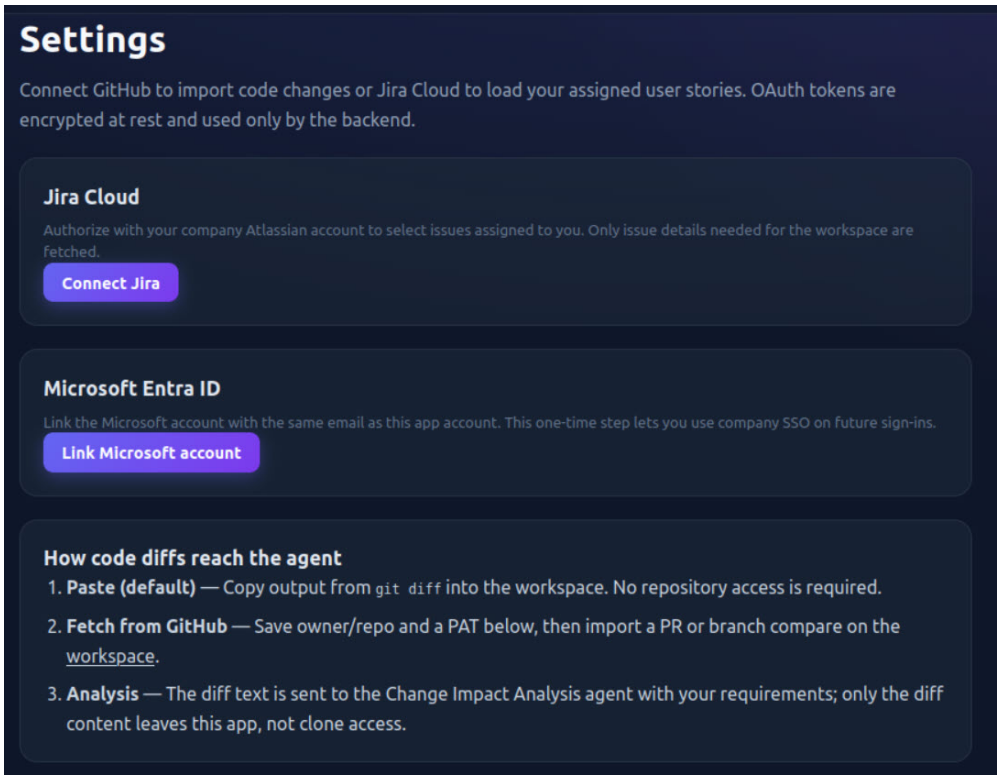


Figure 4.2: Settings page where Jira, GitHub, and sign-in are configured. Availability is contingent upon the configuration made by the administrator and valid credentials.

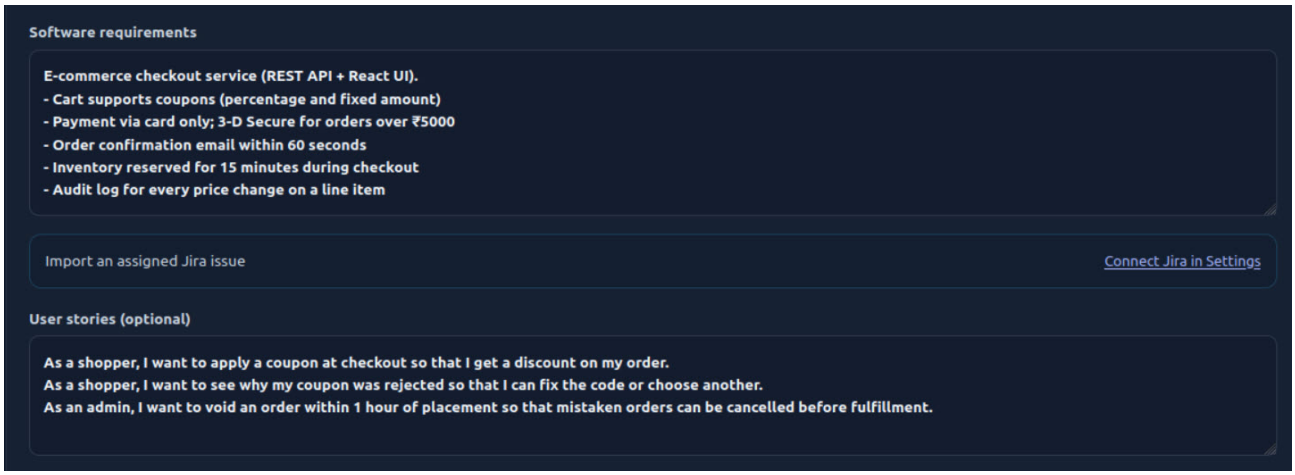


Figure 4.3: Example request input containing quality context and a user story before agent processing.

4. RESULTS AND DISCUSSIONS

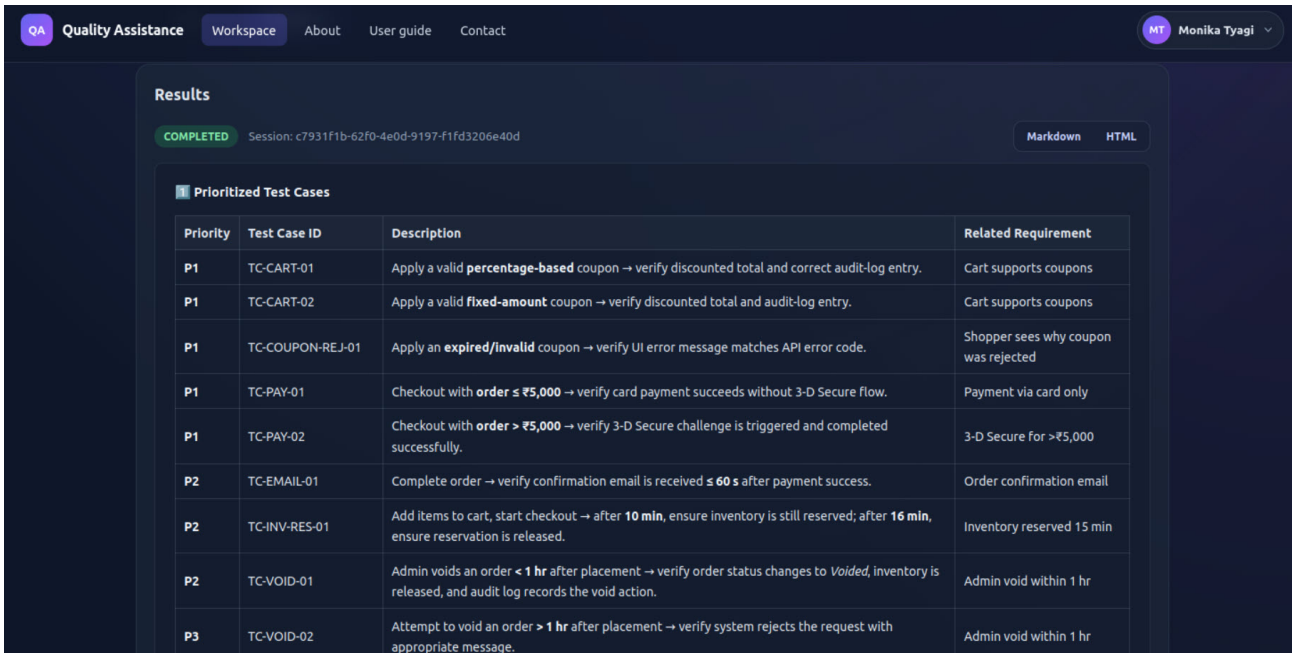


Figure 4.4: Example of the response generated along with test case recommendations.



Figure 4.5: Example of the test data suggested as part of the response generated.

4. RESULTS AND DISCUSSIONS

3 **Automation Notes** (for **test_script_automation** agent)

Area	Automation Approach	Script Outline / Tools
API Tests	Use **Postman/Newman** or **REST-Assured** for end-to-end flow (add to cart → apply coupon → checkout).	• Parameterize coupon codes, payment amounts • Validate response codes & JSON schema • Capture `orderId` for downstream checks
UI Tests	Selenium/WebDriver (or Cypress) for React checkout UI.	• Locate coupon input, submit and assert error/discount message • Mock 3-D Secure redirect via test gateway • Verify email-sent toast and order-status page
Email Verification	Use **MailHog** or **SMTP mock server** to capture outbound emails; assert receipt time.	• Start timer before order finalization • Poll mock server for matching `orderId` email • Assert timestamp ≤ 60 s
Inventory Timer	Background job simulation using **time-travel** in test environment (e.g., `clock` mock).	• After checkout, fast-forward 16 min and query inventory status via API • Verify reservation released
Audit Log Validation	Direct DB query or log-collector API (ELK).	• After price change, query `audit_log` table for recent entry • Assert fields: user, oldPrice, newPrice, timestamp
Admin Void	API call with admin token; combine with UI flow for admin portal.	• Submit void request within 1 hr, assert status `Voided` • Attempt after 1 hr, assert error message
CI/CD Integration	Add to **GitHub Actions** pipeline: 1 Unit → 2 API contract → 3 End-to-end UI (headless Chrome) → 4 Post-run report.	Use environment variables for secrets (payment gateway mock keys).
Self-Healing (future)	Tag UI elements with stable `data-testid` attributes; configure selector fallback strategy.	Include in **self_healing_test_scripts** recommendation later.

Figure 4.6: Example of the automation notes and test script recommendations.

4️⃣ **Release Risks & Mitigations** (for **release_readiness_evaluation**)

Risk	Impact	Likelihood	Mitigation
Coupon mis-calculation (percentage vs fixed)	Wrong discount → revenue loss, poor UX	Medium	End-to-end price verification in every build; unit tests for discount engine
3-D Secure failure for >₹5,000 orders	Payment decline, cart abandonment	High (depends on external gateway)	Mock gateway in staging; monitor real-time success rate after release
Email latency >60 s	Customer confusion, support tickets	Medium	Deploy email service with SLA; add alert on queue depth; include performance test in CI
Inventory reservation timeout bug	Overselling, fulfillment issues	Low to Medium	Automated timer test; log monitoring for "reservation expired" events
Audit log omission	Compliance breach, debugging difficulty	Low	Enforce log write via middleware; unit test audit interceptor
Admin void window logic error	Orders stuck in pending state, financial reversal problems	Medium	Regression suite covering <1 hr and >1 hr cases; code review of time-window logic
Concurrency coupon application	Race condition causing double discount	Low	Stress test with parallel users; lock cart updates at DB level
React UI regression (broken data-testid)	Automation scripts break, manual testing overhead	Medium	Adopt `data-testid` convention; run UI regression smoke after each UI change

Overall Release Readiness – If all **P1** test cases pass in CI, email latency ≤ 45 s in load test, and audit-log coverage is 100 %, the release can be considered **green**. Any failure in P1 or a missed SLA (email, 3-D Secure) should halt promotion to production.

Figure 4.7: Example of release risk and readiness analysis based on the submitted context.

4.4 Evaluation Plan

When I designed the evaluation, I chose to compare three conditions on the same tasks: a person doing the work by hand, one general-purpose LLM given the same inputs, and the full multi-agent application. I included the single-LLM condition to find out whether splitting the work among specialists really helps. The agent service already has a single-agent mode, which it uses for local Ollama models, so the same code can supply that baseline. A comparison with existing commercial tools is a useful extension. I kept the first round to public requirements and open-source repositories or trackers. Any organisational material used later has to be anonymised and has to wait for whatever data-governance approval the organisation needs.

4.4.1 Dataset and Protocol

In the protocol I defined, each project gets a small, closed set of requirements, stories, code changes, and any test-run or defect records that already exist. Expected requirement-to-test links and expert-reviewed reference outputs are written down before the assistant is run. The model, model version, prompt revision, and app configuration stay fixed for a comparison. Because the model does not always give the same answer, each task is repeated where possible, and both the raw output and the review decision are kept. Where practical, the person scoring the artifacts is not told which workflow produced them.

4.4.2 Measures

Measure	Operational definition	Interpretation
Requirement traceability	Percentage of in-scope requirements linked to at least one reviewer-accepted test case.	Tests whether generated artifacts preserve requirement coverage links.
Scenario coverage	Reviewer-rated coverage of agreed positive, negative, boundary, and risk conditions.	Measures breadth and relevance rather than the raw number of generated cases.
Artifact correctness	Proportion of sampled cases judged correct and actionable by independent reviewers.	Captures factual and procedural quality; report agreement between reviewers.
Effort	Active authoring and review time per requirement or change.	A reduction is meaningful only if review and correction time are included.
Defect analysis quality	Precision, recall, or F-score against adjudicated defect categories when labels exist.	Applies only where a suitable labeled reference set is available.
Response time and cost	End-to-end latency and model usage per completed task.	Quantifies operational overhead and helps assess scalability.
User acceptance	Structured ratings of usefulness, clarity, and trust from representative practitioners.	Complements objective measures but should not replace correctness review.

Table 4.2: Proposed evaluation measures; no values are reported because a controlled evaluation has not yet been completed.

4.4.3 End-to-End Time: One Week to One Hour

The practical claim I most want to test is about time. Producing a full set of reviewed quality artefacts for one scenario means test cases, test data, an automation outline, a change-impact note, bug reports for the failures, a test summary, and a release-readiness note. Done by hand, I estimate this takes about a week of working time. With the application, the target is about an hour, and that hour includes several rounds in which an engineer reads the output, gives more guidance, and reruns it.

These two figures are my planning estimate and my target. They are not measured results. Table 4.3 gives the matrix I defined to confirm or reject them.

Metric	Definition	Manual	Assisted	Measured
Cycle time T	Wall-clock time from receiving the requirements to an accepted set of artefacts.	about 1 week	about 1 hour	TBD
Active human effort E	Time a person spends preparing inputs, reading output, and giving instructions.	TBD	TBD	TBD
Rounds N	Number of review-and-rerun rounds before the set is accepted.	—	TBD	TBD
First-draft latency L_1	Time from submitting the request to the first complete response.	—	TBD	TBD
Acceptance rate A	Share of artefacts accepted unchanged or after minor edits.	TBD	TBD	TBD
Time reduction	$(T_{\text{manual}} - T_{\text{assisted}})/T_{\text{manual}} \times 100$, reported only when the accepted artefacts are comparable.	—	target: large	TBD

Table 4.3: Matrix for the one-week to one-hour claim. Manual and assisted entries are estimates or targets; the measured column is to be filled in after the evaluation runs.

The assisted time includes the human review rounds, because a draft produced in ten minutes that takes an hour to fix has not saved anything. An experienced tester first prepares the set by hand, with the time recorded. The same scenario then goes through the assistant with a fixed number of rounds, and a reviewer accepts or rejects each artefact against reference criteria written down beforehand.

4.4.4 Metrics for Each Specialist

The nine specialists do very different jobs, so a single score for all of them would say little. Table 4.4 gives each one its own main metric. Where an agent only suggests something, such

as a repair or an automation approach, I score how good the suggestion is and not whether it worked, because the application does not apply it.

Specialist	Main metric	How it is calculated
S01 Test case generation	Requirement coverage and validity	Requirements covered by at least one accepted case divided by all requirements; valid cases divided by generated cases. The duplicate rate is reported too.
S02 Test data generation	Data validity	Records that satisfy the stated constraints divided by records generated; scenarios supported by the data divided by scenarios required.
S03 Test script automation	Recommendation precision	Recommendations an expert judges appropriate divided by recommendations made.
S04 Test execution	Root-cause accuracy	Failures given the correct cause divided by failures with a known cause (application defect, environment, test script, data, or configuration).
S05 Self-healing scripts	Repair-suggestion accuracy	Suggestions an expert judges correct divided by suggestions made.
S06 Change impact analysis	Precision, recall, and F1	Checked against the tests that are really affected by the change. Precision is correct impacted tests over tests flagged; recall is correct impacted tests over all impacted tests.
S07 Bug report generation	Completeness and acceptance	Reports with all required fields divided by all reports; reports a reviewer accepts divided by all reports; classification accuracy against the known defect type.
S08 Test summary reporting	Factual accuracy	Statements that match the execution data divided by verifiable statements. A finding the data does not support counts as an error.
S09 Release readiness	Decision agreement	Scenarios where the recommendation matches the decision of experienced testers divided by all scenarios. This is decision support, not an approval.

Table 4.4: Proposed main metric for each specialist. No values are reported because the evaluation has not been carried out.

4.4.5 Orchestration and Workflow Metrics

The specialists are only part of the picture. The root agent has to pick the right ones, hand over the right context, and combine the answers, and the engineer has to be able to refine the result. Table 4.5 lists what I defined to measure for the workflow as a whole.

Metric	Definition
Specialist selection accuracy	Requests where the root agent called the expected specialists divided by all requests, checked in the agent logs.
Unnecessary calls	Specialist calls that were not needed divided by all specialist calls.
Workflow completion rate	Requests that ended with a usable response divided by requests started.
Human correction rate	Artefacts the reviewer had to change divided by all artefacts produced.
Repeat consistency	Share of repeated runs that give the same decision or classification.
Cost per workflow	Model usage and cost for one completed request, including the reruns.

Table 4.5: Proposed orchestration and workflow metrics.

4.4.6 Human Evaluation

Some things cannot be scored with a formula, for example whether a test case is meaningful or a root-cause explanation is useful. For these I defined an expert rating step. Two or three experienced testers rate each artefact from 1 (very poor) to 5 (excellent) on correctness, completeness, relevance, clarity, and practical usability, and they are not told which condition produced which artefact. To show the ratings can be trusted, I report how well the reviewers agree, using Cohen's kappa for two reviewers or Fleiss' kappa for more.

4.4.7 Benchmark Dataset

Every comparison needs inputs whose right answers are already known. Table 4.6 lists the pilot set I designed. The sizes are a starting point and can grow if time allows.

Component	Pilot size	Ground truth prepared in advance
Requirements and user stories	10	Expected requirement-to-test links and reviewed reference cases.
Code-change scenarios	10	Source before and after, a description of the change, and the tests that are really affected.
Execution logs	10	The true root cause of each failure.
Defect scenarios	10	Expected type, severity, and required report fields.
Release scenarios	10	Release or no-release decisions made independently by experienced testers.
Test-data scenarios	10	The constraints and boundary values the data must meet.

Table 4.6: Proposed pilot benchmark. The sizes are planning figures.

4.4.8 Analysis and Validity

For numbers I report sample size, the shape of the data, confidence intervals, and which tests I used. I also look at error types, not only averages. On the qualitative side I look for missing cases, claims with no support, unclear requirements, and places where the model and an expert disagree. Model and prompt versions are written down, otherwise a rerun later is hard to explain. With samples this small I kept the statistics simple: medians or means with the spread, precision, recall, and F1 for the identification tasks, and paired comparisons on the same scenarios using the Wilcoxon signed-rank test, or a paired t-test if its assumptions hold, with an effect size such as Cohen's d. Each assisted run is repeated three times, because the output varies, and the variation is reported. If the differences are small or the sample is too small to be sure, the report says so.

The threats I considered are the usual ones. The dataset may not look like the work people actually do. Reviewers will disagree. The model will not give the same answer twice. The provider may change the model under us. Examples may leak into prompts. And a long generated list may simply create more review. A small public-data run would be enough to talk about feasibility. It would not be enough to talk about enterprise workflows.

4.5 Limitations

- If the user gives thin or wrong context, or if the chosen model is weak, the recommendations follow that.
- Routing between agents is still mostly the prompt. I did not add formal schemas or independent quality gates between stages.
- Refinement works by adding instructions and rerunning. A user cannot yet edit or approve individual cases one at a time.
- Session state is in memory. Restart the agent process and the conversation is gone.
- The system does not run external suites, edit scripts, or block a release.
- Retrieval over requirements, test assets, and old defects, and a Jenkins or similar CI hook, are still to be done.
- Tests cover a few settings and services. UI, model quality, load, and a full end-to-end path still need work.
- I have planning notes for production, not a signed-off deployment or an organisation-specific privacy review.

Chapter 5

Conclusions

This project aims to address the gaps between different software quality activities by developing an AI-assisted application that supports various stages of the Software Testing Life Cycle (STLC). The prototype brings together a React and TypeScript frontend, a FastAPI backend, a PostgreSQL database, and a separate agent service built using Google's Agent Development Kit (ADK). At the centre of the system is a root agent that understands incoming requests and directs them to nine specialised agents. Each agent focuses on a particular testing activity, including test-case design, test-data generation, automation suggestions, execution analysis, self-healing recommendations, change-impact analysis, defect reporting, test summaries, and release-readiness assessment.

Along with these capabilities, the application provides user authentication and stores submitted requests for future reference. It also integrates with Jira Cloud to access issue details and GitHub to retrieve code changes. Additional features include optional Microsoft Entra single sign-on and support for configuring different AI model providers.

The prototype shows how multiple testing-related activities can be brought together within a single application and supported through specialised AI agents. However, further evaluation is needed before drawing conclusions about its actual benefits. At this stage, there is no experimental evidence to establish whether the system improves test coverage, reduces the overall effort required by quality engineers, or helps identify more defects. The agents are intended to assist engineers by providing suggestions and organising relevant information, not

by replacing their judgement.

It is also important to understand the limitations of the current implementation. The execution analysis agent reviews available test results but does not run test suites on its own. Similarly, the self-healing agent suggests possible changes to automation scripts but does not apply those changes automatically. The release-readiness assessment provides supporting information for decision-making; the final decision to approve or reject a release remains with the responsible team.

5.1 Future Work

The next steps I have identified, in order:

1. Structured outputs and checks between specialist stages;
2. An explicit approval step and editing of individual cases, building on the refine-and-rerun loop that already exists;
3. An evaluation of traceability, correctness, review effort, latency, and cost against a baseline written down in advance;
4. Trials on public projects and, after approval, on anonymized organisational artifacts;
5. Retrieval over approved requirements, test repositories, and historical defects;
6. A careful CI/CD hook for test execution, and session history that survives a restart and can be audited; and
7. Load, reliability, accessibility, privacy, and security reviews before this is treated as a wider deployment.

I already know an LLM can write something that looks like a test case. The question I still cannot answer is whether this orchestrated assistant produces work that people can trace and use, often enough that the review it creates is still cheaper than doing the task without it. The prototype is the place to run that comparison. Screenshots of a working demo are not the comparison.

References

[1] International Organization for Standardization. *ISO/IEC/IEEE 29119-1:2022: Software and systems engineering—Software testing—Part 1: General concepts*. ISO, 2022. 1, 6

[2] International Software Testing Qualifications Board. Certified tester foundation level syllabus, version 4.0. https://www.istqb.org/certifications/certified-tester-foundation-level, 2023. 1, 6

[3] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*, 50(4):911–936, 2024. doi: 10.1109/TSE.2024.3368208. 1, 8

[4] Monika Tyagi. Quality assistance application: Project source code repository. https://github.com/monikatyagiisc/quality-assistance-app, 2026. Accessed November 2026. 4, 24

[5] Katalon. Katalon true: The AI platform for software quality. https://katalon.com/. Accessed November 2026. 7

[6] testRigor. testrigor: Generative AI-based test automation tool. https://testrigor.com/. Accessed November 2026. 7

[7] Diffblue. Diffblue: Automated unit test generation. https://www.diffblue.com/. Accessed November 2026. 7

REFERENCES

[8] Tonic.ai. Tonic.ai: Synthetic data platform. https://www.tonic.ai/. Accessed November 2026. 7

[9] mabl. mabl: Agentic testing platform. https://www.mabl.com/. Accessed November 2026. 7

[10] Applitools. Applitools visual AI testing platform. https://applitools.com/. Accessed November 2026. 7

[11] Saleema Amershi, Andrew Begel, Christian Bird, Robert De Line, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice*, pages 291–300, 2019. doi: 10.1109/ICSE-SEIP.2019.00042. 8

[12] Aazaade Faraji and Nuno Pombo. AI-driven software test automation: An AI4SE-oriented survey of techniques, tools, and challenges. *IEEE Access*, 13:183296–183313, 2025. doi: 10.1109/ACCESS.2025.3623944. 8

[13] Afonso Fontes and Gregory Gay. The integration of machine learning into automated test generation: A systematic mapping study. *Software Testing, Verification and Reliability*, 33(4):e1845, 2023. doi: 10.1002/stvr.1845. 8, 11

[14] Navid Bin Hasan, Md. Ashraful Islam, Junaed Younus Khan, Sanjida Senjik, and Anindya Iqbal. Automatic high-level test case generation using large language models. In *International Conference on Mining Software Repositories (MSR)*, 2025. arXiv:2503.17998. 8

[15] Mariam Lahami and Moez Krichen. Architectural perspectives on GenAI-driven software testing systems: A systematic literature review. *Information and Software Technology*, 199:108246, 2026. doi: 10.1016/j.infsof.2026.108246. 9

[16] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. 9

REFERENCES

[17] Lei Wang, Cheng Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wei Zhao, Zhe Wei, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024. 9

[18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. 9

[19] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, U.S. Department of Commerce, 2023. 10

[20] Sebastián Ramírez. Fastapi documentation. https://fastapi.tiangolo.com/. Accessed November 2026. 17

[21] Google. Agent development kit documentation. https://google.github.io/adk-docs/. Accessed November 2026. 17

[22] Model Context Protocol. What is the model context protocol (MCP)? https://modelcontextprotocol.io/introduction. Accessed November 2026. 21